



Project Report

On

Digital Logic Gate Simulator using OOP in C++

Course No: CSE 2100

Course Name: Object-Oriented Programming Laboratory

Submitted By-

Mrinmoy Sarker

Roll: 2309042

19 May , 2026

Department of Electronics and Communication Engineering

Khulna University of Engineering & Technology

Khulna-9203, Bangladesh.

Introduction:

Background →

Digital logic gates serve as the fundamental building blocks of modern digital electronics, processing binary inputs to execute essential computational operations. In computer science education, understanding the behavior of these gates—ranging from basic nodes like AND and OR to universal elements like NAND and NOR—is critical. While physical hardware testing provides hands-on experience, software-based simulation offers a risk-free, highly accessible environment to observe and validate logical operations instantly.

Motivation →

The core **motivation** behind this project is to bridge the gap between theoretical digital logic and practical software implementation. By designing a **Digital Logic Gate Simulator**, complex hardware abstractions are converted into functional software entities. Utilizing Object-Oriented Programming (OOP) principles in C++ provides an excellent opportunity to mirror real-world modular electronics. Concepts such as **inheritance** allow specific gates to inherit attributes from a common base class, cleanly replicating how physical integrated circuits (ICs) share foundational electrical properties.

Objectives →

The primary goals of this project are:

- **To Develop a Functional Simulator:** Build a lightweight, interactive console application that accurately simulates the truth tables of primary, universal, and exclusive logic gates.
- **To Apply Core OOP Concepts:** Successfully implement class hierarchies, constructors for real-time simulation updates, and **destructors** to cleanly transition between operations.
- **To Enhance Code Reusability:** Utilize a centralized base class (LogicGate) to hold shared variables and state parameters, minimizing redundancy across derived gate classes.

Project Overview

This report documents the Object-Oriented Programming concepts implemented in the Digital Logic Gate Simulator written in C++. The simulator supports eight logic gates — AND, OR, NOT, Buffer, NAND, NOR, XOR, and XNOR — through an interactive console menu.

Each gate is modeled as a C++ class derived from a common base class, making the project a comprehensive demonstration of core OOP principles. The sections below identify each concept, explain what it means, and show exactly where it appears in the code.

Concepts at a glance →

Sl.	OOP Concept	Where implemented in the Project
01	Classes	LogicGate, AndGate, OrGate, NotGate, etc.
02	Objects	myAnd, myOr, myNot ... in switch-case blocks
03	Hierarchical Inheritance	8 gate classes inherit from LogicGate
04	Encapsulation	protected members x, y, z, gateName
05	Polymorphism (Runtime)	virtual ~LogicGate() in base class
06	Operator Overloading	operator>() in each gate class
07	Constructors	Parameterized + initializer list in each class
08	Destructors	~GateName() prints status on object end-of-life
09	Abstract Base Class Pattern	LogicGate as structural blueprint

Classes:

A class is a user-defined blueprint that bundles data (attributes) and functions (methods) into a single unit. It acts as a template for creating objects.

In this project, LogicGate serves as the base class, and each gate type (AND, OR, NOT, etc.) is its own derived class. Each class encapsulates the gate's behavior inside its own operator() method.

```
// Base Class
class LogicGate {
protected:
    bool x, y, z;
    string gateName;
public:
    LogicGate(string name) : gateName(name) {
        cout << "\nSimulation for --> " << gateName << endl;
    }
    virtual ~LogicGate() {} // Added virtual destructor for safe inheritance
};
```

Objects:

An object is a runtime instance of a class. It occupies memory and uses the class blueprint to hold its own state and invoke its methods.

In main(), a new gate object (e.g., myAnd, myOr) is instantiated inside each switch case. Each object lives just for that iteration — demonstrating object lifecycle within a local scope.

```
cout << "Enter Input B (0 or 1): "; cin >> b;
AndGate myAnd;
// Using overloaded operator()
cout << "\n-->>> Result: " << myAnd(a, b) << endl;
break;
```

Inheritance (Hierarchical):

Inheritance allows a derived class to acquire properties and behaviors from a base class. Hierarchical inheritance means multiple derived classes share one common base class.

All eight gate classes (AndGate, OrGate, NotGate, Buffer, NandGate, NorGate, XorGate, XnorGate) inherit from the single base class LogicGate, forming a hierarchical inheritance tree.

```

// Base Class
class LogicGate {
protected:
    bool x, y, z;
    string gateName;
public:
    LogicGate(string name) : gateName(name) {
        cout << "\nSimulation for --> " << gateName << endl;
    }
    virtual ~LogicGate() {} // Added virtual destructor for safe inheritance
};

// 1. AND Gate
class AndGate : public LogicGate {
public:
    AndGate() : LogicGate("AND Gate Operation") {}

    // Operator overloading for binary inputs
    bool operator()(bool x, bool y) {
        z = x && y;
        return z;
    }

    ~AndGate() { cout << "Opt for next operation\n"; }
};

// 2. OR Gate

```

Encapsulation:

Encapsulation is the bundling of data and methods inside a class, and restricting direct access to internal data through access specifiers (private, protected, public).

The member variables x, y, and z are declared protected in LogicGate, preventing direct external access while still allowing derived classes to use them. The gate logic is hidden inside operator() methods.

```

class LogicGate {
protected:
    bool x, y, z;
    string gateName;
public:
    LogicGate(string name) : gateName(name) {
        cout << "\nSimulation for --> " << gateName << endl;
    }
    virtual ~LogicGate() {}
}

```

Polymorphism(Runtime):

Polymorphism means 'many forms'. Runtime polymorphism (via virtual functions) lets derived class methods override base class behavior, resolved at runtime through a base class pointer.

The virtual destructor in LogicGate enables runtime polymorphism. When a derived gate object is deleted through a base class pointer, the correct derived destructor is called — preventing memory leaks.

```
class LogicGate {
protected:
    bool x, y, z;
    string gateName;
public:
    LogicGate(string name) : gateName(name) {
        cout << "\nSimulation for --> " << gateName << endl;
    }
    virtual ~LogicGate() {}
};

// 1. AND Gate
class AndGate : public LogicGate {
public:
    AndGate() : LogicGate("AND Gate Operation") {}

    // Operator overloading for binary inputs
    bool operator()(bool x, bool y) {

~AndGate() { cout << "Opt for next operation\n"; }
};
```

Operator Overloading:

Operator overloading lets you redefine the behavior of built-in C++ operators for user-defined types. The function call `operator operator()` makes objects callable like functions.

Each gate class overloads `operator()` to accept input signals and return the gate's boolean output. This makes gate objects intuitive to use — `myAnd(a, b)` reads just like a real logic gate evaluation.

```

// 3. NOT Gate
class NotGate : public LogicGate {
public:
    NotGate() : LogicGate("NOT (Inverter) Gate Operation") {}

    // Operator overloading for a single input
    bool operator() (bool x) {
        z = !x;
        return z;
    }

    ~NotGate() { cout << "Opt for next operation\n"; }
};

// 4. Buffer
class Buffer : public LogicGate {
public:
    Buffer() : LogicGate("Buffer Operation") {}

    bool operator() (bool x) {
        z = x;
        return z;
    }

    ~Buffer() { cout << "Opt for next operation\n"; }
};

```

Constructors:

A constructor is a special member function that is automatically called when an object is created. It initializes the object's state. A parameterized constructor accepts arguments; a constructor initializer list passes values to base class constructors.

LogicGate has a parameterized constructor that sets gateName. Each derived class constructor uses an initializer list to pass the gate's name string up to the base class constructor.

```

class LogicGate {
protected:
    bool x, y, z;
    string gateName;
public:
    LogicGate(string name) : gateName(name) {
        cout << "\nSimulation for --> " << gateName << endl;
    }
    virtual ~LogicGate() {}
};

// 1. AND Gate
class AndGate : public LogicGate {
public:
    AndGate() : LogicGate("AND Gate Operation") {}
}

```

Destructors:

A destructor is a special member function called automatically when an object goes out of scope or is explicitly deleted. It performs cleanup. A virtual destructor in a base class ensures correct destruction through polymorphism.

Each derived gate class has a destructor that prints 'Opt for next operation', serving as a notification when the gate object's lifetime ends. The base class has a virtual destructor for safe polymorphic deletion.

```

        cout << "\nSimulation for --> " << gateName << endl;
    }
    virtual ~LogicGate() {}
};

// 1. AND Gate
class AndGate : public LogicGate {
public:
    AndGate() : LogicGate("AND Gate Operation") {}

    // Operator overloading for binary inputs
    bool operator()(bool x, bool y) {
        z = x && y;
        return z;
    }

    ~AndGate() { cout << "Opt for next operation\n"; }
};

```


Abstract Base Class Pattern:

An abstract base class defines a common interface for all derived classes without providing complete implementation itself. While not purely abstract here (no pure virtual functions), LogicGate plays this structural role.

LogicGate holds the shared data (gateName, x, y, z) and the virtual destructor, but provides no compute logic — that belongs entirely to derived classes. This enforces a clean separation between structure and behavior.

```
// Base Class
class LogicGate {
protected:
    bool x, y, z;
    string gateName;
public:
    LogicGate(string name) : gateName(name) {
        cout << "\nSimulation for --> " << gateName << endl;
    }
    virtual ~LogicGate() {}
};
```

Code:

```
#include <iostream>

#include <string>

using namespace std;

// Base Class

class LogicGate {
protected:
    bool x, y, z;
    string gateName;
public:
    LogicGate(string name) : gateName(name) {
```

```

cout << "\nSimulation for --> " << gateName << endl;

}

virtual ~LogicGate() {}

};

// 1. AND Gate

class AndGate : public LogicGate {

public:

    AndGate() : LogicGate("AND Gate Operation") {}

// Operator overloading for binary inputs

    bool operator()(bool x, bool y) {

        z = x && y;

        return z;

    } ~AndGate() { cout << "Opt for next operation\n"; }

};

// 2. OR Gate

class OrGate : public LogicGate {

public:

    OrGate() : LogicGate("Or Gate Operation") {}

bool operator()(bool x, bool y) {

    z = x || y;

    return z;

}

~OrGate() { cout << "Opt for next operation\n"; }

};

// 3. NOT Gate

class NotGate : public LogicGate {

public:

    NotGate() : LogicGate("NOT (Inverter) Gate Operation") {}

// Operator overloading for a single input

    bool operator()(bool x) {

```

```

        z = !x;

        return z;

    }

~NotGate() { cout << "Opt for next operation\n"; }

};

// 4. Buffer

class Buffer : public LogicGate {

public:

    Buffer() : LogicGate("Buffer Operation") {}

    bool operator()(bool x) {

        z = x;

        return z;

    }

    ~Buffer() { cout << "Opt for next operation\n"; }

};

// 5. NAND Gate

class NandGate : public LogicGate {

public:

    NandGate() : LogicGate("NAND (Universal Gate) Operation") {}

    bool operator()(bool x, bool y) {

        z = !(x && y);

        return z;

    }

    ~NandGate() { cout << "Opt for next operation\n"; }

};

// 6. NOR Gate

class NorGate : public LogicGate {

public:

    NorGate() : LogicGate("NOR (Universal Gate) Operation") {}

    bool operator()(bool x, bool y) {

```

```

        z = !(x || y);

        return z;

    }

~NorGate() { cout << "Opt for next operation\n"; }

};

// 7. XOR Gate

class XorGate : public LogicGate {

public:

    XorGate() : LogicGate("XOR (Exclusive Gate) Operation") {}

    bool operator()(bool x, bool y) {

        z = (x != y);

        return z;

    }

~XorGate() { cout << "Opt for next operation\n"; }

};

// 8. XnorGate

class XnorGate : public LogicGate {

public:

    XnorGate() : LogicGate("XNOR (Exclusive Gate) Operation") {}

    bool operator()(bool x, bool y) {

        z = !(x != y);

        return z;

    }

~XnorGate() { cout << "Opt for next operation\n"; }

};

int main() {

    int option;

    bool a, b;

    while (true) {

```

```

    cout << "\n===== LOGIC GATE SIMULATOR =====" << endl;

    cout << "\n1. AND Gate\n2. OR Gate\n3. NOT Gate(single input)\n4. Buffer (single input)\n5. NAND Gate\n6. NOR Gate\n7. XOR Gate\n8. XNOR Gate\n0. Exit" << endl;

    cout << "Select: ";

    cin >> option;

    if (option == 0) break;

    if (option < 0 || option > 8) {

        cout << "Invalid choice!" << endl;

        continue;

    }cout << "Enter Input A (0 or 1): ";

    cin >> a;

    switch (option) {

        case 1: {

            cout << "Enter Input B (0 or 1): "; cin >> b;

            AndGate myAnd;

            // Using overloaded operator()

            cout << "\n-->>> Result: " << myAnd(a, b) << endl;

            break;

        }

        case 2: {

            cout << "Enter Input B (0 or 1): "; cin >> b;

            OrGate myOr;

            cout << "\n-->>> Result: " << myOr(a, b) << endl;

            break;

        }

        case 3: {

            NotGate myNot;

            cout << "\n-->>> Result: " << myNot(a) << endl;

            break;

        }

        case 4: {

```

```

    Buffer myBuf;

    cout << "\n-->>> Result: " << myBuf(a) << endl;

    break;
}

case 5: {

    cout << "Enter Input B (0 or 1): "; cin >> b;

    NandGate myNand;

    cout << "\n-->>> Result: " << myNand(a, b) << endl;

    break;
}

case 6: {

    cout << "Enter Input B (0 or 1): "; cin >> b;

    NorGate myNor;

    cout << "\n-->>> Result: " << myNor(a, b) << endl;

    break;
}

case 7: {

    cout << "Enter Input B (0 or 1): "; cin >> b;

    XorGate myXor;

    cout << "\n-->>> Result: " << myXor(a, b) << endl;

    break;
}

case 8: {

    cout << "Enter Input B (0 or 1): "; cin >> b;

    XnorGate myXnor;

    cout << "\n-->>> Result: " << myXnor(a, b) << endl;

    break;
}
}

}cout << "\nExiting the simulation ..." << endl;

```

```
    return 0;
}
```

Output Example:

```
===== LOGIC GATE SIMULATOR =====
```

```
1. AND Gate
2. OR Gate
3. NOT Gate(single input)
4. Buffer (single input)
5. NAND Gate
6. NOR Gate
7. XOR Gate
8. XNOR Gate
0. Exit
```

```
Select: 2
```

```
Enter Input A (0 or 1): 1
```

```
Enter Input B (0 or 1): 0
```

```
Simulation for --> Or Gate Operation
```

```
-->>> Result: 1
```

```
Opt for next operation
```

```
Enter Input A (0 or 1): 1
```

```
Simulation for --> NOT (Inverter) Gate Operation
```

```
-->>> Result: 0
```

```
Opt for next operation
```

```
Enter Input A (0 or 1): 1
```

```
Enter Input B (0 or 1): 0
```

```
Simulation for --> NAND (Universal Gate) Operation
```

```
-->>> Result: 1
```

```
Opt for next operation
```

```
Enter Input A (0 or 1): 1
```

```
Enter Input B (0 or 1): 0
```

```
Simulation for --> XOR (Exclusive Gate) Operation
```

```
-->>> Result: 1
```

```
Opt for next operation
```

```
===== LOGIC GATE SIMULATOR =====
```

```
1. AND Gate
2. OR Gate
3. NOT Gate(single input)
4. Buffer (single input)
5. NAND Gate
6. NOR Gate
7. XOR Gate
8. XNOR Gate
0. Exit
```

```
Select: 1
```

```
Enter Input A (0 or 1): 1
```

```
Enter Input B (0 or 1): 0
```

```
Simulation for --> AND Gate Operation
```

```
-->>> Result: 0
```

```
Opt for next operation
```

```
Enter Input A (0 or 1): 1
```

```
Simulation for --> Buffer Operation
```

```
-->>> Result: 1
```

```
Opt for next operation
```

```
Enter Input A (0 or 1): 1
```

```
Enter Input B (0 or 1): 0
```

```
Simulation for --> NOR (Universal Gate) Operation
```

```
-->>> Result: 0
```

```
Opt for next operation
```

```
Enter Input A (0 or 1): 1
```

```
Enter Input B (0 or 1): 0
```

```
Simulation for --> XNOR (Exclusive Gate) Operation
```

```
-->>> Result: 0
```

```
Opt for next operation
```

Conclusion:

The Digital Logic Gate Simulator effectively demonstrates nine fundamental OOP concepts through a focused, real-world inspired design. The hierarchical inheritance structure — eight gate classes under one base class — is the backbone of the project, while operator overloading gives each gate an intuitive interface.

Protected encapsulation keeps internal gate state safe, constructors with initializer lists ensure proper initialization across the hierarchy, and the virtual destructor enables correct polymorphic cleanup. Together, these concepts show how OOP principles translate directly into clean, maintainable C++ code.